

安全部署至 Cloud Run

來源：<https://codelabs.developers.google.com/secure-cloud-run-deployment?hl=zh-tw>

步驟數：7

安全部署至 Cloud Run 的基本做法

目錄

1. [1. 總覽](#)
2. [2. 設定和需求](#)
3. [3. 查看 README](#)
4. [4. 安全地部署服務](#)
5. [5. 進行已驗證的要求](#)
6. [6. 驗證程式與驗證使用者](#)
7. [7. 恭喜！](#)

步驟 1 · 約 0 分鐘

1. 總覽

您將修改預設步驟，將服務部署至 Cloud Run，以提升安全性，然後瞭解如何以安全的方式存取已部署的應用程式。這個應用程式是 Cymbal Eats 應用程式的「合作夥伴註冊服務」，與 Cymbal Eats 合作的公司會使用這個應用程式處理餐點訂單。

學習目標

只要對將應用程式部署至 Cloud Run 的預設步驟稍做調整，就能大幅提升安全性。您將使用現有的應用程式和部署說明，變更部署步驟，以提升已部署應用程式的安全性。

接著，您會瞭解如何授權存取應用程式及提出授權要求。

這並非應用程式部署安全性的完整說明，而是介紹您可對日後所有應用程式部署作業進行的變更，以極少的力氣提升安全性。

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' button, which is circled in blue. Below this, the 'Select a project' section includes a search bar. The 'New Project' section follows, with a 'Project name' field containing 'my-kubernetes-codelab', also circled in blue. Below the name field, the 'Project ID' is displayed as 'my-kubernetes-codelab' with a note that it cannot be changed later. The 'Location' is set to 'No organization'. At the bottom, there are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新該位置資訊。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生不重複的字串，通常您不需要在意這個字串。在大多數程式碼研究室中，您需要參照專案 ID (通常會標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間都會維持這個設定。
- 請注意，部分 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是專屬 ID，選定後就無法變更，且其他使用者無法使用。您是該 ID 的唯一使用者。即使刪除專案，該 ID 也無法再使用。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用應該不高，甚至完全免費。如要關閉資源，避免產生本教學課程以外的費用，您可以刪除自己建立的資源，或刪除整個專案。Google

Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟用 Cloud Shell

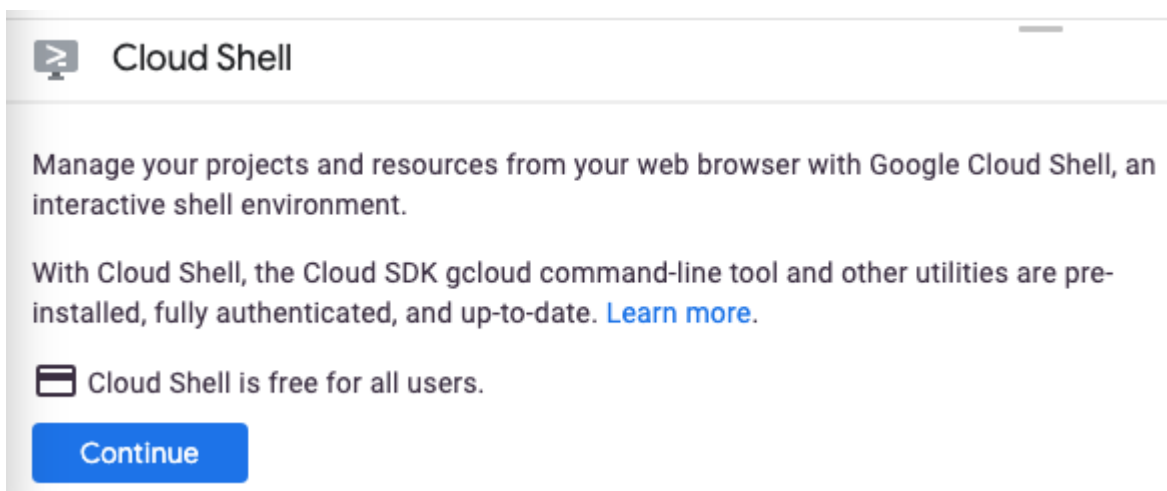
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



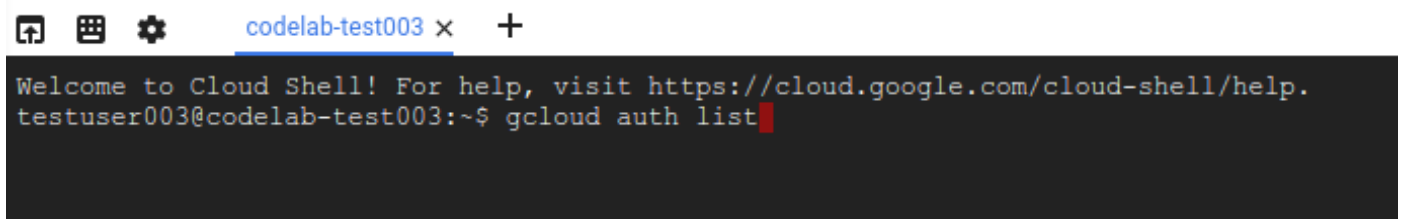
。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面 (位於需捲動位置)，說明這個指令列環境。點選「繼續」後，這則訊息日後就不會再出現。以下是這個初次畫面的樣子：



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器搭載您需要的所有開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器或 Chromebook 上完成。

連線至 Cloud Shell 後，您應會發現自己通過驗證，且專案已設為您的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

```
Credentialed Accounts
ACTIVE ACCOUNT
*          <my_account>@<my_domain.com>

To set the active account, run:
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

環境設定

在本實驗室中，您將在 Cloud Shell 指令列中執行指令。通常可以直接複製並貼上指令，但有時需要將預留位置值變更為正確值。

1. 設定專案 ID 的環境變數，以供後續指令使用：

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION=us-central1
export SERVICE_NAME=partner-registration-service
```

2. 啟用將執行應用程式的 Cloud Run 服務 API、提供 NoSQL 資料儲存空間的 Firestore API、部署指令使用的 Cloud Build API，以及用於保存建構應用程式容器的 Artifact Registry：

```
gcloud services enable \  
  run.googleapis.com \  
  firestore.googleapis.com \  
  cloudbuild.googleapis.com \  
  artifactregistry.googleapis.com
```

3. 以原生模式初始化 Firestore 資料庫。該指令會使用 App Engine API，因此必須先啟用。

指令必須指定 App Engine 的區域 (我們不會使用，但基於歷史因素必須建立)，以及資料庫的區域。我們會為 App Engine 使用 us-central，並為資料庫使用 **nam5**。**nam5** 是美國多區域位置。多地區位置可盡可能提高資料庫的可用性和耐用性。

```
gcloud services enable appengine.googleapis.com  
  
gcloud app create --region=us-central  
gcloud firestore databases create --region=nam5
```

4. 複製範例應用程式存放區並前往目錄

```
git clone https://github.com/GoogleCloudPlatform/cymbal-eats.git  
  
cd cymbal-eats/partner-registration-service
```

步驟 3 · 約 0 分鐘

3. 查看 README

開啟編輯器並查看組成應用程式的檔案。查看 **README.md**，其中說明部署這個應用程式所需的步驟。其中有些步驟可能涉及要考慮的隱含或明確安全決策。您將變更其中幾項選擇，以提升已部署應用程式的安全性，詳情請參閱：

步驟 3 - 執行 `npm install`

瞭解應用程式中使用的任何第三方軟體的來源和完整性非常重要。管理軟體供應鏈安全與建構任何軟體 (不只是部署至 Cloud Run 的應用程式) 都有關。本實驗室著重於部署作業，因此不會探討這個領域，但您不妨另外研究這個主題。

步驟 4 和 5 - 編輯及執行 `deploy.sh`

這些步驟會將應用程式部署至 Cloud Run，並將大部分選項設為預設值。您將修改這個步驟，透過以下兩種主要方式提升部署作業的安全性：

1. 請勿允許未經驗證的存取權。在探索期間，允許這麼做可能很方便，但這是供商業合作夥伴使用的網路服務，因此一律應驗證使用者身分。
2. 指定應用程式必須使用專屬服務帳戶，且該帳戶只具備必要權限，而非預設帳戶 (預設帳戶可能具備超出需求的 API 和資源存取權)。這就是所謂的**最小權限原則**，也是應用程式安全性的基本概念。

步驟 6 至 11 - 提出範例網頁要求，驗證行為是否正確

由於應用程式部署現在需要驗證，這些要求必須包含要求者身分的證明。您不會變更這些檔案，而是直接從指令列發出要求。

步驟 4 · 約 0 分鐘

4. 安全地部署服務

我們在 `deploy.sh` 指令碼中發現兩項必要變更：不允許未經驗證的存取權，以及使用權限最低的專屬服務帳戶。

您必須先建立新的服務帳戶，然後編輯 `deploy.sh` 指令碼，參照該服務帳戶並禁止未經驗證的存取權，接著執行修改後的指令碼來部署服務，才能執行修改後的 `deploy.sh` 指令碼。

建立服務帳戶，並授予存取 Firestore/Datastore 的必要權限

```
gcloud iam service-accounts create partner-sa

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:partner-sa@${PROJECT_ID}.iam.gserviceaccount.com" \
  --role=roles/datastore.user
```

編輯「`deploy.sh`」

修改 `deploy.sh` 檔案，禁止未經驗證的存取權(`--no-allow-unauthenticated`)，並為已部署的應用程式指定新的服務帳戶(`--service-account`)。將 `GOOGLE_PROJECT_ID` 更正為您專案的 ID。

您將刪除前兩行，並變更其他三行，如下所示。

```
gcloud run deploy $SERVICE_NAME \
  --source . \
  --platform managed \
  --region ${REGION} \
  --no-allow-unauthenticated \
  --project=$PROJECT_ID \
  --service-account=partner-sa@${PROJECT_ID}.iam.gserviceaccount.com
```

部署服務

在指令列執行 `deploy.sh` 指令碼：

```
./deploy.sh
```

部署完成後，指令輸出內容的最後一行會顯示新應用程式的服務網址。將網址儲存在環境變數中：

```
export SERVICE_URL=<URL from last line of command output>
```

現在請嘗試使用 `curl` 工具從應用程式擷取訂單：

```
curl -i -X GET $SERVICE_URL/partners
```

`curl` 指令的 `-i` 旗標會告知指令在輸出內容中加入回應標頭。輸出內容的第一行應為：

```
HTTP/2 403
```

部署應用程式時，選擇**禁止**未經驗證的要求。這個 curl 指令不含驗證資訊，因此遭到 Cloud Run 拒絕。實際部署的應用程式甚至不會執行，也不會從這項要求接收任何資料。

步驟 5 · 約 0 分鐘

5. 進行已驗證的要求

您必須先驗證網路要求，Cloud Run 才會允許這些要求叫用已部署的應用程式。如要驗證網路要求，請加入以下形式的 `Authorization` 標頭：

```
Authorization: Bearer identity-token
```

身分識別權杖是由可信的驗證供應商核發，是經過加密簽署的短期編碼字串。在此情況下，您必須提供 Google 核發的有效身分識別權杖，且權杖不得過期。

以使用者帳戶提出要求

Google Cloud CLI 工具可為預設已驗證使用者提供權杖。執行這項指令，取得您帳戶的身分識別權杖，並儲存至 `ID_TOKEN` 環境變數：

```
export ID_TOKEN=$(gcloud auth print-identity-token)
```

根據預設，Google 核發的身分識別權杖有效期限為一小時。執行下列 `curl` 指令，發出先前因未獲授權而遭拒的要求。這項指令會包含必要的標頭：

```
curl -i -X GET $SERVICE_URL/partners \  
-H "Authorization: Bearer $ID_TOKEN"
```

指令輸出內容的開頭應為 `HTTP/2 200`，表示要求可接受且正在處理中。(如果等待一小時後再次嘗試這項要求，就會失敗，因為權杖已過期)。回應主體位於輸出內容的結尾，空白行之後：

```
{"status": "success", "data": []}
```

目前沒有任何合作夥伴。

使用目錄中的範例 JSON 資料，透過兩項 `curl` 指令註冊合作夥伴：

```
curl -X POST \  
-H "Authorization: Bearer $ID_TOKEN" \  
-H "Content-Type: application/json" \  
-d "@example-partner.json" \  
$SERVICE_URL/partner
```

和

```
curl -X POST \  
-H "Authorization: Bearer $ID_TOKEN" \  
-H "Content-Type: application/json" \  
-d "@example-partner2.json" \  
$SERVICE_URL/partner
```

重複先前的 GET 要求，即可查看所有已註冊的合作夥伴：

```
curl -i -X GET $SERVICE_URL/partners \
-H "Authorization: Bearer $ID_TOKEN"
```

您應該會看到內容多得多的 JSON 資料，其中提供兩個已註冊合作夥伴的相關資訊。

以未經授權的帳戶提出要求

上一個步驟中經過驗證的要求之所以成功，不僅是因為經過驗證，也是因為經過驗證的使用者 (您的帳戶) 獲得授權。也就是說，該帳戶有權叫用應用程式。並非所有通過驗證的帳戶都有權這麼做。

先前要求中使用的預設帳戶已獲得授權，因為該帳戶建立了含有應用程式的專案，且預設有權叫用帳戶中的任何 Cloud Run 應用程式。如有需要，可以撤銷這項權限，這在正式版應用程式中是理想做法。您現在不會這麼做，而是要建立新的服務帳戶，且不指派任何權限或角色，然後使用該帳戶嘗試存取已部署的應用程式。

1. 建立名為 `tester` 的服務帳戶。

```
gcloud iam service-accounts create tester
```

2. 您會以與先前取得預設帳戶身分權杖大致相同的方式，取得這個新帳戶的身分權杖。不過，這需要預設帳戶具備模擬服務帳戶的權限。授予帳戶這項權限。

```
export USER_EMAIL=$(gcloud config list account --format "value(core.account)")

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="user:$USER_EMAIL" \
  --role=roles/iam.serviceAccountTokenCreator
```

3. 現在請執行下列指令，將這個新帳戶的身分識別權杖儲存在 `TEST_IDENTITY` 環境變數中。如果指令顯示錯誤訊息，請稍候一到兩分鐘，然後再試一次。

```
export TEST_TOKEN=$( \
  gcloud auth print-identity-token \
    --impersonate-service-account \
    "tester@$PROJECT_ID.iam.gserviceaccount.com" \
)
```

4. 如常發出已驗證的網頁要求，但使用這個身分識別權杖：

```
curl -i -X GET $SERVICE_URL/partners \
-H "Authorization: Bearer $TEST_TOKEN"
```

由於要求已通過驗證，但未獲授權，因此指令輸出內容會再次以 `HTTP/2 403` 開頭。新的服務帳戶沒有叫用這個應用程式的權限。

授權帳戶

使用者或服務帳戶必須擁有 Cloud Run 服務的 Cloud Run 叫用者角色，才能向該服務提出要求。使用下列指令，將該角色授予測試人員服務帳戶：

```
export REGION=us-central1
gcloud run services add-iam-policy-binding ${SERVICE_NAME} \
  --member="serviceAccount:tester@$PROJECT_ID.iam.gserviceaccount.com" \
  --role=roles/run.invoker \
  --region=${REGION}
```

等待一到兩分鐘，讓新角色更新完畢，然後重複經過驗證的要求。如果自首次儲存 TEST_TOKEN 後已過一小時以上，請儲存新的 TEST_TOKEN。

```
curl -i -X GET $SERVICE_URL/partners \
  -H "Authorization: Bearer $TEST_TOKEN"
```

指令輸出內容現在會以 `HTTP/1.1 200 OK` 開頭，最後一行則包含 JSON 回應。這項要求已由 Cloud Run 接受，並由應用程式處理。

6. 驗證程式與驗證使用者

您目前提出的已驗證要求都使用 `curl` 指令列工具。您也可以使用其他工具和程式設計語言。不過，您無法使用網頁瀏覽器和純網頁發出經過驗證的 Cloud Run 要求。如果使用者點選連結或按鈕，在網頁中提交表單，瀏覽器不會新增 Cloud Run 驗證要求所需的 `Authorization` 標頭。

Cloud Run 的內建驗證機制適用於程式，不適用於使用者。

注意：

Cloud Run 可以代管面向使用者的網頁應用程式，但這類應用程式必須將 Cloud Run 設為允許來自使用者網頁瀏覽器的未經驗證要求。如果應用程式需要使用者驗證，則必須自行處理，而不是要求 Cloud Run 執行驗證。應用程式可以透過與 Cloud Run 外部網頁應用程式相同的方式執行這項操作。具體做法不在本程式碼研究室的範圍內。

您可能已注意到，到目前為止，範例要求的相關回應都是 JSON 物件，而非網頁。這是因為合作夥伴註冊服務是供程式使用，而 JSON 格式方便程式取用。接著，您將編寫及執行程式，以取用及使用這項資料。

Python 程式傳送的已驗證要求

程式可透過標準 HTTP 網頁要求，對受保護的 Cloud Run 應用程式發出已驗證的要求，但必須包含 `Authorization` 標頭。這些程式唯一的新挑戰，就是取得有效且未過期的身分識別權杖，並放在該標頭中。Cloud Run 會使用 Google Cloud Identity and Access Management (IAM) 驗證該權杖，因此權杖必須由 IAM 認可的授權單位核發及簽署。許多語言都有可供程式使用的用戶端程式庫，可要求核發這類權杖。本範例使用的用戶端程式庫是 Python `google.auth` 程式庫。一般來說，您可以使用多種 Python 程式庫發出網路要求，這個範例使用的是熱門的 `requests` 模組。

首先，請安裝下列兩個用戶端程式庫：

```
pip install google-auth
pip install requests
```

以下是要求預設使用者身分識別權杖的 Python 程式碼：

```
credentials, _ = google.auth.default()
credentials.refresh(google.auth.transport.requests.Request())
identity_token = credentials.id_token
```

如果您使用指令殼層 (例如 Cloud Shell 或自有電腦上的標準終端機殼層)，預設使用者是已在該殼層中完成驗證的使用者。在 Cloud Shell 中，這通常是登入 Google 的使用者。在其他情況下，則是透過 `gcloud auth login` 或其他 `gcloud` 指令驗證的使用者。如果使用者從未登入，系統不會有預設使用者，因此這段程式碼會失敗。

如果某個程式向另一個程式提出要求，您通常不會想使用人員身分，而是要求程式的身分。這時候服務帳戶就能派上用場。您部署 Cloud Run 服務時，使用專屬服務帳戶提供身分，以便在發出 API 要求時使用，例如對 Cloud Firestore 發出要求。程式在 Google Cloud 平台執行時，用戶端程式庫會自動使用指派給程式的服務帳戶做為預設身分，因此相同的程式碼在這兩種情況下都能運作。

以下是使用 Python 程式碼，透過新增的 `Authorization` 標頭提出要求：

```
auth_header = {"Authorization": "Bearer " + identity_token}
response = requests.get(url, headers=auth_header)
```

下列完整的 Python 程式會向 Cloud Run 服務發出已通過驗證的要求，擷取所有已註冊的合作夥伴，然後列印其名稱和指派的 ID。複製並執行下列指令，將這段程式碼儲存到 `print_partners.py` 檔案。

```
cat > ./print_partners.py << EOF
def print_partners():
    import google.auth
    import google.auth.transport.requests
    import requests

    credentials, _ = google.auth.default()
    credentials.refresh(google.auth.transport.requests.Request())
    identity_token = credentials.id_token

    auth_header = {"Authorization": "Bearer " + identity_token}
    response = requests.get("${SERVICE_URL}/partners", headers=auth_header)

    parsed_response = response.json()
    partners = parsed_response["data"]

    for partner in partners:
        print(f"{partner['partnerId']}: {partner['name']}")

print_partners()
EOF
```

您將使用殼層指令執行這個程式。您必須先以預設使用者身分進行驗證，程式才能使用這些憑證。執行下列 `gcloud auth` 指令：

```
gcloud auth application-default login
```

按照操作說明完成登入。然後從指令列執行程式：

```
python print_partners.py
```

輸出內容應如下所示：

```
10102: Zippy food delivery
67292: Foodful
```

程式的要求已送達 Cloud Run 服務，因為程式已透過您的身分驗證，而您是這個專案的擁有者，因此預設有權執行程式。這個程式通常會以服務帳戶的身分執行。在大多數 Google Cloud 產品 (例如 Cloud Run 或 App Engine) 上執行時，預設身分會是服務帳戶，而非個人帳戶。

步驟 7 · 約 0 分鐘

7. 恭喜！

恭喜，您已完成本程式碼研究室！

後續步驟：

探索其他 Cymbal Eats 程式碼研究室：

- [使用 Eventarc 觸發 Cloud Workflows](#)
- [透過 Cloud Storage 觸發事件處理作業](#)
- [從 Cloud Run 連線至私人 Cloud SQL](#)
- [從 Cloud Run 連線至全代管資料庫](#)
- [使用 Identity-Aware Proxy \(IAP\) 保護無伺服器應用程式](#)
- [使用 Cloud Scheduler 觸發 Cloud Run 工作](#)
- [保護 Cloud Run 輸入流量](#)
- [從 GKE Autopilot 連線至私人 AlloyDB](#)

清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本教學課程所用資源的費用，請刪除含有相關資源的專案，或者保留專案但刪除個別資源。

刪除專案

如要避免付費，最簡單的方法就是刪除您為了本教學課程所建立的專案。
